REPUBLIC OF IRAQ • MINISTRY OF HIGHER EDUCATION & SCIENTIFIC RESEARCH
MIDDLE TECHNICAL UNIVERSITY
ELECTRICAL AND ELECTRONIC TECHNICAL COLLEGE (EETC)
Department of Cybersecurity Engineering Technology (CSTE)

UNDERGRADUATE COURSEWARE • LECTURE NOTES

WEB APPLICATIONS SECURITY

ADVANCED VULNERABILITY ASSESSMENT, THREAT MITIGATION & DEFENSE

Course Code: CSTE4107 | Credits: 5 ECTS | Total Workload: 125 Hours
Type: Elective / Core Specialization

Prepared in Full Accordance with the Bologna Process • Approved Module Description Form

COURSE CLASSIFICATION

Module Specifications

Level • Semester: **Level 4 • Semester 7**
Delivery Mode: **Theory • Laboratory • Seminar**
Structured Workload: **79 hrs/sem (5 hrs/week)**
Academic Year: **2026 – 2027**

COURSE LECTURER

Lecturer Ahmad Salim

Department: **Cybersecurity Engineering Tech. (CSTE)**
Institution: **Middle Technical University (MTU)**
Role: **Course Lecturer & Academic Instructor**

ahmadsalim@mtu.edu.iq

Week 1: Introduction to Web Application Security

Learning Objectives

By the end of this lecture, you should be able to:

- *Explain what "web application security" means and how it differs from general network security.*
- *Trace the full connection path between a browser and a server: DNS, TCP, TLS, then HTTP — and say why each layer matters.*
- *Distinguish between client-side and server-side attack surfaces in a web application.*
- *Apply the CIA triad (Confidentiality, Integrity, Availability) to a web application scenario.*
- *Explain why mixing code and data in the same input stream is the root cause of injection-class vulnerabilities.*
- *Describe the legal and ethical framework for security testing: authorization, rules of engagement, and responsible disclosure.*

What is a web application?

A web application is software that runs on a remote server and is accessed through a browser, Gmail, online banking, a university portal, an online store. Unlike a program installed on your laptop, none of the application's code runs permanently on your machine. Instead, your browser sends a request over the internet, the server processes it and often talks to a database behind the scenes, and it sends back a response that your browser turns into the page you see. This request/response cycle happens every time you click a link, submit a form, or load a page.

How a browser and a server actually connect

Before any application-level request happens, several lower layers do their job first. It's worth naming them now, because next week's lecture (HTTP Protocol Deep Dive) builds directly on this sequence:

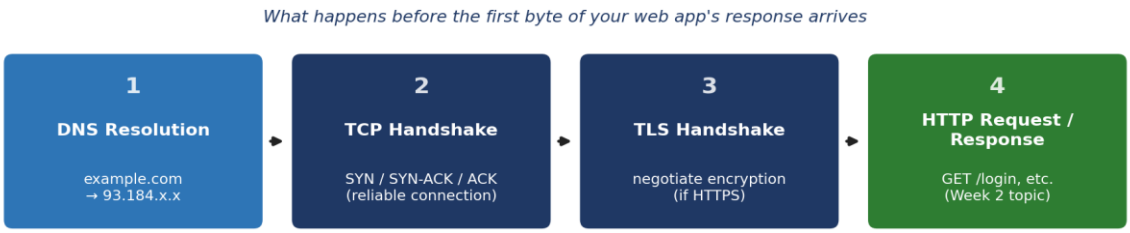


Figure 1 — Four steps happen before your browser ever "talks" to the application itself.

- *DNS Resolution: the browser turns a name like example.com into an IP address.*
- *TCP Handshake: the browser and server exchange SYN / SYN-ACK / ACK to open a reliable connection.*
- *TLS Handshake: (for HTTPS) both sides agree on encryption keys so traffic in between can't be read or silently altered. TLS provides confidentiality and integrity for data in transit and, through certificates, authenticates the server to the client.*
- *HTTP Request/Response: only now does the actual application conversation begin (GET /login, and so on).*

Web application security assumes this connection layer already happened correctly, and focuses on what occurs after it inside the HTTP conversation itself. That's exactly where next week's lecture picks up.

Network security vs. web application security

Network security protects the pipes: firewalls, routers, VPNs, the TLS layer you just saw. Web application security protects what runs on top of those pipes the application logic itself. A firewall won't stop an attacker who logs in through the normal front door and then manipulates a form field to see another user's data, because from the network's point of view that traffic looks completely legitimate. That's an application-layer problem, and it's what this course focuses on: not how data travels, but what the application does with it once it arrives.

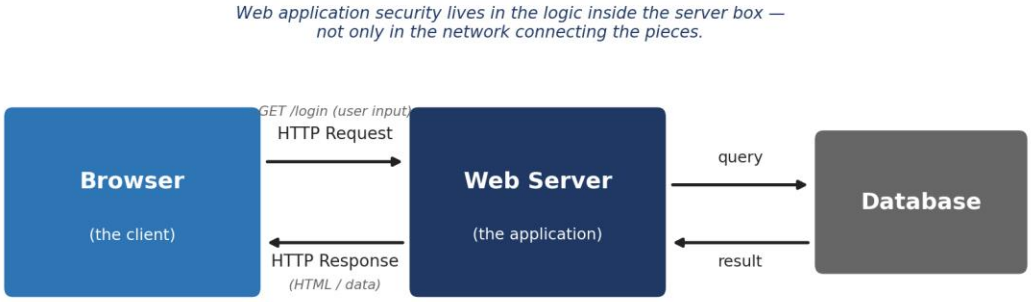


Figure 2 — The basic application flow. Most vulnerabilities you'll study live inside the server logic, not the wires connecting the boxes.

The attack surface of a web application

It helps to split where attacks actually land into two broad categories. This split will keep reappearing as a way to organize everything you learn this semester:

Client-Side Attacks	Server-Side Attacks
(target the victim's browser)	**(target the app server, database, and backend services)**
Cross-Site Scripting (XSS) — Week 6	SQL Injection — Week 9
Cross-Site Request Forgery (CSRF) — Week 7	Command Injection
DOM-based manipulation	Server-Side Request Forgery (SSRF)
Clickjacking	Broken Access Control — Week 12

Client-side attacks trick the victim's own browser into doing something harmful, often without the server ever being touched directly. Server-side attacks go after the backend directly: the application server, its database, or the internal services it talks to knowing which side an attack targets helps you reason about its impact and where the fix has to go.

Why web apps are attractive targets

Three properties make web applications a favorite target for attackers:

- *They're publicly reachable. There's no need to breach an internal network first, anyone with a browser can reach the login page.*
- *They usually hold valuable data: passwords, payment details, personal records, business data.*
- *They're complex. More features and more code mean more places for a small mistake to hide, and a single missed check can undermine an otherwise well-built system.*

The CIA triad, applied to web apps

Almost every security discussion, including this course, comes back to three goals:

- *Confidentiality: only authorized users can see the data. Your bank balance should stay private to you.*
- *Integrity: data isn't changed without permission. No one should be able to quietly edit your order total or someone else's grade.*
- *Availability: the service stays reachable. A flood of fake requests shouldn't be able to take the site offline for real users.*

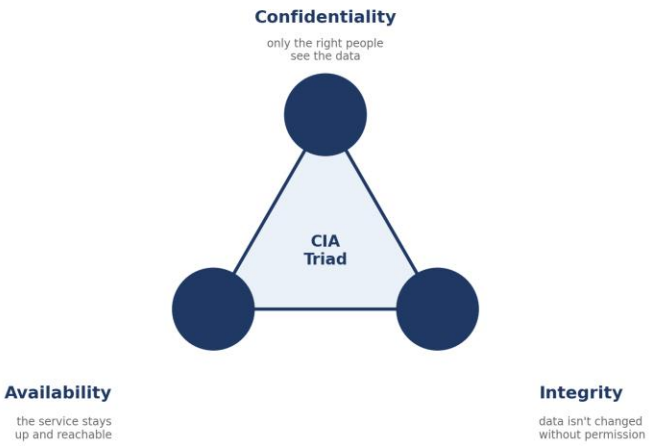


Figure 3 — The CIA triad. Every attack you study this semester breaks at least one of these three goals.

You'll use this triad as a lens all semester. XSS mainly threatens confidentiality by stealing session data. SQL injection can break both confidentiality and integrity at once. A denial-of-service attack targets availability specifically.

The fundamental flaw: mixing code and data

Almost every injection-class vulnerability: SQL Injection, Command Injection, XSS, and others you'll meet this semester traces back to one single design mistake: letting user input be interpreted as executable instructions instead of treating it purely as raw data. A safely built system always keeps a hard boundary between "this is a command" and "this is just a value." The moment that boundary blurs, whoever controls the input effectively controls part of the program's logic. Keep this one sentence in mind, it explains more of the OWASP Top 10 than any other single idea:

> *"An interpreter that treats user input as code, instead of as data, is already broken."*

A detailed example: SQL Injection

Consider a login form that checks a username and password against a database. A safely written application treats whatever the user types purely as data. But if the developer builds the database query by directly pasting in the raw text the user typed, the boundary described above disappears an attacker can type something that changes the query itself, not just its value.

Nothing about the network or the server's operating system was attacked here, the vulnerability lives entirely in how the application handled user input. This exact idea,

SQL Injection, is the subject of Week 9, and the same root cause reappears in XSS (Week 6), CSRF (Week 7), and several other topics this semester.

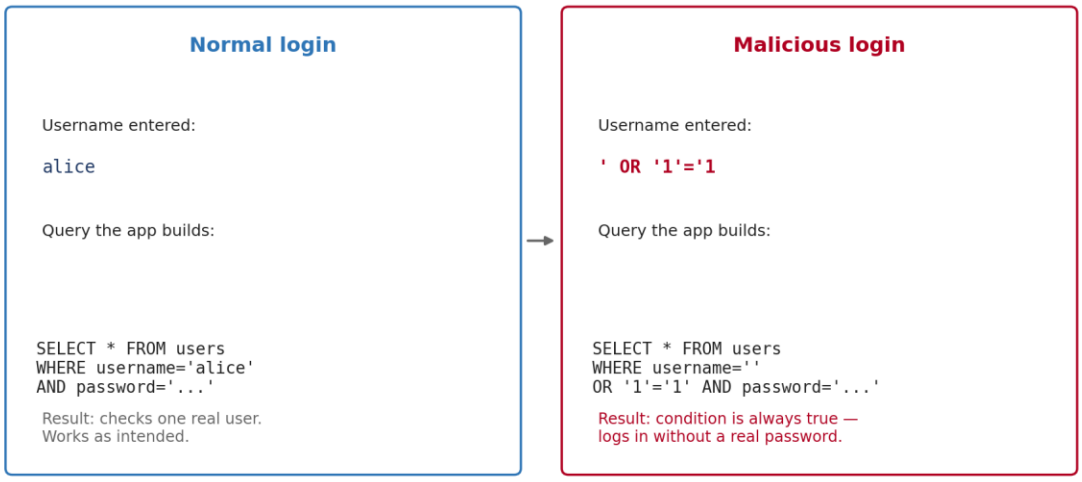


Figure 4 — Same login form, two different inputs. The malicious input turns part of the query into a condition that's always true.

The OWASP Top 10 — and which edition to use

Different sources may show a different ranking depending on which edition they cite, so the year matters. The edition most textbooks, exam question banks, and the official OWASP documentation listed in this course's required texts are built around is the 2021 edition:

- *A01:2021 – Broken Access Control*
- *A02:2021 – Cryptographic Failures*
- *A03:2021 – Injection*
- *A04:2021 – Insecure Design*
- *A05:2021 – Security Misconfiguration*
- *A06:2021 – Vulnerable and Outdated Components*
- *A07:2021 – Identification and Authentication Failures*
- *A08:2021 – Software and Data Integrity Failures*
- *A09:2021 – Security Logging and Monitoring Failures*
- *A10:2021 – Server-Side Request Forgery (SSRF)*

OWASP released an updated 2025 edition that reshuffles this ranking (for example, Security Misconfiguration jumps from #5 to #2, and Software Supply Chain Failures appears as a new category). This course will primarily reference the 2021 edition, since

it matches your required text, but when you read outside material, always check which year is being cited before comparing category numbers across sources.

Legal and ethical responsibility

Knowing how to find a vulnerability comes with a responsibility not to misuse that knowledge. Testing, scanning, or attempting to break into a system you don't own and don't have explicit written permission to test is illegal in most countries, regardless of good intentions: "I was just curious" is not a legal defense. Professional security work is built around three concepts you should know by name:

- *Rules of Engagement (RoE): a written agreement, made before testing starts, that defines exactly what is in-scope (systems you may test) and out-of-scope (everything else), plus timing, methods allowed, and points of contact.*
- *Responsible / Coordinated Vulnerability Disclosure: when you find a real flaw, you report it privately to the system's owner first, giving them a fair window (commonly around 90 days) to fix it before any public disclosure.*
- *Safe Harbor & Bug Bounty Programs: platforms such as HackerOne and Bugcrowd let organizations publish an official, legally protected scope for researchers, so testing within that scope is authorized and (often) paid, instead of being a crime.*

Throughout this course, every hands-on exercise happens inside environments built specifically for practice, so you can experiment freely without legal or ethical risk. Get comfortable with this mindset now permission first, always because it's the standard you're expected to carry into your professional career.

Week 2: HTTP Protocol Deep Dive

Learning Objectives

By the end of this lecture, you should be able to:

- *Describe the structure of an HTTP request and response: start line, headers, and body.*
- *Explain the common HTTP methods and when each is appropriate — and how the method itself can be tampered with.*
- *Distinguish precisely between 401 Unauthorized (unauthenticated) and 403 Forbidden (unauthorized), and explain what status codes can leak.*
- *Explain why HTTP is stateless, how cookies restore state, and what each SameSite value (Strict / Lax / None) actually does.*
- *Identify key security-relevant HTTP headers, including how Referer and caching headers can leak sensitive data.*
- *Explain how TLS turns HTTP into HTTPS, connecting back to the connection layers from Week 1.*
- *Recognize Host Header Injection as a way HTTP's own trust assumptions get abused.*

Anatomy of an HTTP message

Every HTTP message, request or response, has the same four-part shape: a start line, a set of headers, a blank line, and an optional body. Learn to read this shape and you can read any HTTP traffic you'll ever intercept with a tool like Burp Suite or the browser's DevTools.

- *Start line / status line — what is being asked for (request), or what happened (response).*
- *Headers — metadata about the message: format, size, cookies, caching rules, and more.*
- *Blank line — marks the end of headers; nothing subtle here, just a required separator.*
- *Body — the actual payload: form data going up, HTML/JSON coming back down.*

The method states the client's intent. Two properties matter for security and correctness: whether a method is safe (doesn't change server state) and whether it's idempotent (sending the same request twice has the same effect as sending it once).



Figure 1 — A request and a matching response, broken into their four parts.

HTTP methods

Method	Purpose	Safe?	Idempotent?
GET	Retrieve a resource	Yes	Yes
POST	Create a resource / submit data	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Partially update a resource	No	No
DELETE	Remove a resource	No	Yes
HEAD	Like GET, but headers only, no body	Yes	Yes
OPTIONS	Ask what methods/headers are allowed (CORS preflight)	Yes	Yes

One practical security consequence: GET parameters travel in the URL, which means they get written into browser history, server access logs, and proxy logs in plain sight. Never put passwords, tokens, or other secrets in a GET query string, use POST with a body instead.

The method is also something an attacker can try to abuse directly. Verb tampering (or method override abuse) happens when an access-control rule only checks the literal HTTP method; for example, a firewall rule that blocks POST to /admin/delete but forgets GET or HEAD to the same path. Some frameworks also honor a header like X-HTTP-Method-Override, letting a client claim a request is really a PUT or DELETE. If

the application trusts that override without re-checking permissions, an attacker can smuggle a blocked action through an allowed method. The fix is simple in principle: authorize the action, not the verb string.

HTTP status codes at a glance

- *1xx Informational: rare in everyday use, e.g. 100 Continue.*
- *2xx Success: 200 OK, 201 Created.*
- *3xx Redirection: 301 Moved Permanently, 302 Found.*
- *4xx Client Error: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found.*
- *5xx Server Error: 500 Internal Server Error, 503 Service Unavailable.*

401 and 403 are commonly confused, and the exact distinction is worth knowing precisely: 401 Unauthorized actually means unauthenticated, the server doesn't know who you are yet, because no valid credentials (login, token) were presented. 403 Forbidden means the opposite problem: your identity is known and confirmed, but your permissions aren't sufficient for this specific resource. In short, 401 asks "who are you?" and 403 says "I know who you are, and the answer is still no."

Status codes can leak more than they should in other ways too. If a login page returns one message for "wrong password" and a different one for "no such user," an attacker can use that difference to enumerate valid usernames, one status code at a time. A well-designed authentication endpoint returns the same generic error either way. We'll return to this exact issue in Week 3 (Authentication and Session Management).

HTTP is stateless

By design, HTTP treats every request as brand new: the server keeps no memory of your previous requests. That's what "stateless" means. But a login session obviously needs to persist across many requests, so the application has to manufacture that memory itself, and the standard way to do it is cookies.

Three cookie attributes matter enormously for security, and you'll see all three again later this semester:

- *Secure: the cookie is only ever sent over HTTPS, never plain HTTP.*
- *HttpOnly: JavaScript cannot read the cookie, which limits what a successful XSS attack (Week 6) can steal.*
- *SameSite: restricts whether the cookie is sent along with requests coming from another site, which is a core defense against CSRF (Week 7).*

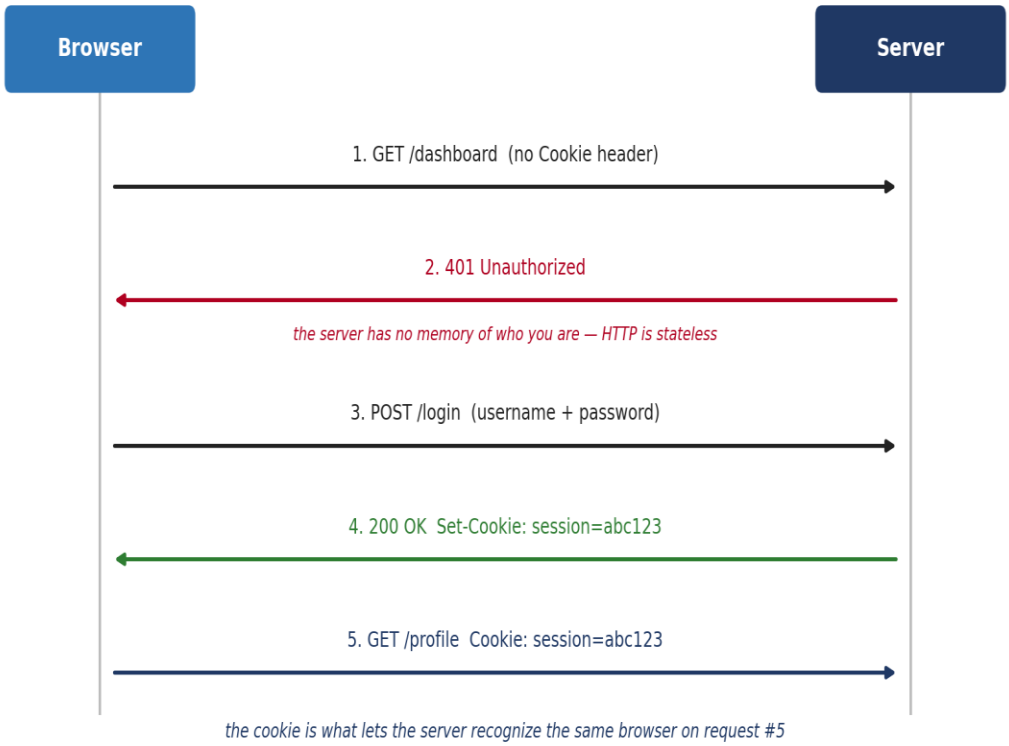


Figure 2 — The server has no memory of its own; the cookie the browser keeps re-sending is what creates the illusion of a continuous session.

SameSite specifically takes one of three values, and the difference matters a lot once you reach CSRF in Week 7:

Value	Behavior
Strict	The cookie is never sent with any cross-site request, even a normal link click coming from another site. Strongest protection, but can break legitimate cross-site flows.
Lax	The cookie is sent only with "safe" top-level navigation (e.g., clicking a normal link), but not with cross-site POSTs, images, or iframes. This is the modern browser default.
None	The cookie is sent with every cross-site request, with no restriction. Requires the Secure flag to also be set (HTTPS only).

- *Domain: controls which hosts receive the cookie.*
- *Path: restricts the URL paths where the cookie is sent.*

Security-relevant HTTP headers

Beyond cookies, a handful of response headers let a server tell the browser to defend itself:

Header	What it protects against
Strict-Transport-Security (HSTS)	Forces the browser to use HTTPS only for this site, blocks downgrade / SSL-stripping attacks.
Content-Security-Policy (CSP)	Restricts which scripts and resources are allowed to run on the page, a major defense against XSS (Week 6).
X-Content-Type-Options: nosniff	Stops the browser from guessing a file's type, closing off some content-type confusion attacks.
X-Frame-Options	Blocks the page from being loaded inside another site's <iframe> prevents clickjacking.
Access-Control-Allow-Origin	Part of CORS: controls which other origins may read this response via JavaScript (Week 14).
Referrer-Policy: no-referrer	Stops the browser from sending the previous page's full URL (which may contain a sensitive token or ID) to a third-party site.

Data leakage through headers and caching

Two everyday headers cause real-world data leaks that have nothing to do with a classic "attack", just careless defaults. The *Referer* header tells the next site which page you came from, including the full URL. If a sensitive URL contains a token (for example, https://app.edu/reset?token=abc123) and that page has a link or embedded image from a third-party site, the browser will happily hand that token to the third party in the Referer header. The Referrer-Policy header (shown above) is how a developer prevents that leak.

Cache-Control: no-store matters for a related but different reason: it tells both the browser and any intermediate proxy never to store a copy of the response at all. Without it, a sensitive page: a bank statement, a grade report: can remain in the browser's local cache after the user logs out. On a shared or public workstation, the next person can sometimes recover that page by pressing "back" or browsing the cache directly, even though the session itself has ended.

Host Header Injection

The Host header tells the server which website the request is meant for, which matters when one server hosts multiple sites. The problem is that some applications trust this

client-supplied header blindly and use it to build absolute URLs: most dangerously, in password-reset emails ("click here: https://{Host}/reset?token=..."). If the app doesn't validate Host against an allow-list of its real domains, an attacker can submit a password-reset request with Host: attacker.com. The application, trusting the header, builds the reset link using that fake domain, so when the legitimate user (or an automated system) clicks it, the reset token is sent straight to the attacker. This specific abuse is known as password-reset poisoning, and the fix is simple: never build security-sensitive URLs from client-controlled input.

HTTPS = HTTP + TLS

HTTPS isn't a different protocol from HTTP, it's the same HTTP conversation, wrapped inside the TLS encryption layer from Week 1's connection sequence. TLS gives you three guarantees: the traffic is encrypted (confidentiality), it can't be silently altered in transit (integrity), and through the site's certificate you have some assurance you're actually talking to the real server (authenticity). That's why a browser's "connection not secure" warning should never be dismissed reflexively: it usually means one of those three guarantees just failed.